Propuesta de Proyecto Práctico

Cliente 360: una red neuronal multi-tarea para decisiones de marketing en e-commerce

Marketing Digital y Publicidad Digital
Grado en Ciencia e Ingeniería de Datos

Fabio Nesta Arteaga Cabrera
Javier Bolívar García-Izquierdo
Pablo Cabeza Lantigua
Jaime Ercilla Martín
Pablo Herrera González

1 Planteamiento

Proponemos construir, en Python, una aplicación de *Deep Learning* que resuelva de forma conjunta tres de las aplicaciones de Machine Learning en marketing digital trabajadas en la asignatura: la predicción del valor futuro del cliente (*Customer Lifetime Value*), la segmentación de clientes y los sistemas de recomendación de productos.

El punto de partida no es entrenar tres modelos independientes, sino **una única red neuronal multi-tarea** que aprende una representación numérica (*embedding*) de cada cliente y de cada producto a partir de su historial de compras, y reutiliza esa misma representación para responder a las tres preguntas de negocio. En nuestro caso, el mismo embedding aprendido por un único modelo pasa a alimentar tres decisiones de marketing distintas, en lugar de necesitar tres pipelines de datos separados.

Además del modelo, el proyecto se plantea como un **producto completo**: una aplicación organizada en servicios independientes (datos, modelo, segmentación, evaluación y panel) que permita a un responsable de marketing consultar para cualquier cliente su valor esperado, su segmento y los productos que conviene recomendarle.

Las preguntas a resolver durante el desarrollo del trabajo son:

1. ¿Mejora una red multi-tarea a los modelos clásicos?
2. ¿Aporta el aprendizaje conjunto una ventaja real frente a entrenar cada tarea por separado?
3. ¿Producen los embeddings segmentos más útiles para marketing que una segmentación clásica?

2 Dataset

Utilizaremos el **Brazilian E-Commerce Public Dataset by Olist**, más de 100.000 pedidos reales y anonimizados, realizados entre 2016 y 2018 en el marketplace brasileño Olist, distribuidos en 9 archivos CSV relacionados entre sí.

De los 9 archivos disponibles, el proyecto utiliza los siguientes cinco:

Archivo	Papel en el proyecto
`olist_customers_dataset.csv`	Identifica al cliente único (`customer_unique_id`), necesario para reconstruir el historial de compras de una misma persona a través de distintos pedidos.
`olist_orders_dataset.csv`	Fecha y estado de cada pedido; permite definir el *target* de CLV y realizar una separación train/test temporal, más realista que una partición aleatoria.
`olist_order_items_dataset.csv`	Relación pedido–producto; es la tabla que construye la matriz de interacciones cliente-producto que alimenta la cabeza de recomendación.
`olist_order_payments_dataset.csv`	Importe pagado por pedido; es la base para calcular el gasto acumulado y futuro de cada cliente (CLV).
`olist_products_dataset.csv`	Categoría y atributos del producto; enriquece el embedding de producto más allá de un identificador.

Se descartan `olist_order_reviews_dataset.csv` (orientado a análisis de sentimiento, fuera del alcance de este proyecto), `olist_sellers_dataset.csv` y `olist_geolocation_dataset.csv` (no aportan información necesaria para las tres tareas seleccionadas), y se contempla el uso opcional de `product_category_name_translation.csv` únicamente para traducir las categorías de producto y facilitar la lectura de resultados en la memoria.

2.1 Retos conocidos del dataset

Antes de modelar, identificamos tres características de Olist que condicionan el diseño y que se tratarán de forma explícita en el proyecto:

- **Baja recurrencia de compra**: la gran mayoría de los clientes realiza un único pedido. Un CLV definido como gasto futuro tendrá por tanto muchos valores cero y una distribución muy asimétrica. Se compararán varias definiciones del *target* (gasto futuro, probabilidad de recompra y gasto transformado con logaritmo) y se justificará la elegida.
- **Alta dispersión de la matriz cliente-producto**: hay miles de productos con muy pocas compras. Para mitigarlo, el embedding de producto se enriquece con su categoría y atributos, y se evalúa también una versión del recomendador a nivel de categoría.
- **Periodo cerrado y desbalanceado**: los datos abarcan 2016–2018, con un volumen muy bajo al inicio y al final. Se acotará el periodo útil y se usará una separación temporal para evitar fugas de información del futuro.

3 Relación con el contenido de la asignatura

La siguiente tabla vincula cada componente del proyecto con las aplicaciones de Machine Learning en marketing digital y los modelos "habituales" vistos en la asignatura, frente a la alternativa de Deep Learning que proponemos:

Aplicación	Modelo habitual (asignatura)	Nuestra propuesta (Deep Learning)
Customer Lifetime Value	Regression, Random Forest, Gradient Boosting	Cabeza de regresión de la red, entrenada sobre el embedding de cliente aprendido conjuntamente con las otras dos tareas.
Sistemas de recomendación	Collaborative Filtering, Matrix Factorization	Neural Collaborative Filtering: cabeza de clasificación que combina embedding de cliente y de producto para estimar la probabilidad de compra.
Segmentación de clientes	K-Means, DBSCAN, clustering jerárquico	K-Means aplicado sobre los embeddings de cliente ya aprendidos por la red, sin necesidad de entrenar un tercer modelo independiente.

4 Arquitectura del modelo

La red se implementará en Python PyTorch, con el siguiente diseño:

1. **Capas de embedding**: una para `cliente_id` y otra para `producto_id`, entrenadas a partir del historial de compras. El embedding de producto se complementa con la categoría del producto y otros atributos, para aliviar el problema de los productos poco comprados.

2. **Cabeza de CLV** (regresión): capas densas sobre el embedding de cliente, con función de pérdida de error cuadrático medio (MSE). Se evaluarán variantes con *target* transformado (logaritmo) o con una cabeza adicional que estime la probabilidad de recompra.

3. **Cabeza de recomendación** (clasificación): combina embedding de cliente y de producto para estimar la probabilidad de compra, entrenada con *negative sampling* sobre los pares cliente-producto sin interacción, con entropía cruzada binaria como función de pérdida.

4. **Entrenamiento conjunto** con una pérdida combinada:

$$\mathcal{L} = \mathcal{L}_{\text{CLV}} + \lambda \cdot \mathcal{L}_{\text{recomendación}}$$

   donde λ pondera la contribución relativa de cada tarea al entrenamiento. El valor de λ no se fija a priori, sino que se explora experimentalmente.

5. **Segmentación posterior**: extracción de los embeddings de cliente ya entrenados y aplicación de K-Means, con visualización mediante PCA o t-SNE para su interpretación. El número de segmentos se elige con criterios cuantitativos y cualitativos.

4.1 Arquitectura de software

El sistema se organiza en cinco servicios independientes que se comunican mediante API REST y comparten los datos pesados (tablas procesadas, pesos y embeddings) a través de un volumen común. Todos se despliegan con Docker, de forma que el conjunto se levanta con una única orden.

Servicio	Responsabilidad
Servicio de datos	Lectura y unión de las cinco tablas, limpieza, construcción de la matriz cliente-producto, variables por cliente, *target* de CLV y separación temporal.
Servicio del modelo	Definición, entrenamiento e inferencia de la red multi-tarea; expone el CLV predicho, las recomendaciones y los embeddings.
Servicio de segmentación	K-Means sobre los embeddings, proyección a dos dimensiones y etiquetado de segmentos.
Servicio de evaluación	Entrena los modelos clásicos de referencia y calcula las métricas comparativas.
Pasarela y panel	Punto de entrada único a los demás servicios y panel interactivo en Streamlit.

5 Evaluación experimental

La evaluación es una parte central del proyecto. Todos los modelos se comparan sobre la misma separación temporal de train/validación/test, con semillas fijas para garantizar la reproducibilidad.

Tarea	Modelos comparados	Métricas
CLV	Media histórica, regresión lineal, Random Forest, Gradient Boosting, cabeza de la red	MAE, RMSE, correlación de Spearman y *lift* del decil superior de clientes
Recomendación	Productos más populares, *Collaborative Filtering* / *Matrix Factorization*, cabeza de la red	Precision@K, Recall@K, NDCG@K y cobertura del catálogo
Segmentación	K-Means sobre variables RFM, K-Means sobre embeddings	Coeficiente de silueta, estabilidad entre ejecuciones e interpretabilidad de los perfiles

Además de la comparación con los modelos clásicos, se realizarán los siguientes experimentos adicionales:

- **Estudio de ablación**: red multi-tarea frente a dos redes entrenadas cada una con una sola tarea, para comprobar si el aprendizaje conjunto aporta una mejora real.
- **Sensibilidad a hiperparámetros**: efecto de λ, de la dimensión del embedding, de la regularización y del número de negativos por muestra positiva.
- **Arranque en frío**: rendimiento con clientes y productos con muy poco historial.
- **Simulación de impacto de negocio**: estimación de cuánto mejoraría una campaña si se dirigiera a los clientes de mayor CLV predicho o con las recomendaciones de la red, frente a una selección aleatoria.

6 Plan de trabajo (6 semanas)

Semana	Hitos
1	**Datos y exploración.** Descarga y unión de las cinco tablas de Olist seleccionadas. Limpieza y análisis exploratorio (distribución de pedidos por cliente, de importes y de categorías). Construcción de la matriz cliente-producto y de las variables agregadas por cliente. Definición del *target* de CLV (ventana de observación y de predicción) y separación temporal de train/validación/test. Creación del repositorio y reparto de servicios.
2	**Modelos de referencia.** Ingeniería de variables RFM. Implementación de los modelos clásicos: media histórica, regresión, Random Forest y Gradient Boosting para CLV; popularidad y *Collaborative Filtering / Matrix Factorization* para recomendación. Fijación del protocolo de evaluación y de las métricas. Puesta en marcha del servicio de datos.
3	**Red multi-tarea (primera versión).** Implementación en PyTorch o Keras de las capas de embedding, la cabeza de CLV y la cabeza de recomendación con *negative sampling*. Entrenamiento conjunto con la pérdida combinada y primeras curvas de aprendizaje. Incorporación de la categoría del producto al embedding. Servicio del modelo con los primeros *endpoints*.
4	**Optimización y ablación.** Búsqueda de hiperparámetros (λ, dimensión del embedding, regularización, negativos por positivo). Comparación de definiciones del *target* de CLV. Estudio de ablación (multi-tarea frente a tareas por separado) y análisis del arranque en frío. Primera tabla comparativa de resultados frente a los modelos clásicos.
5	**Segmentación e interpretación.** Extracción de embeddings y segmentación con K-Means, con selección del número de segmentos. Comparación con la segmentación basada en RFM. Visualización con PCA o t-SNE y perfilado de cada segmento (valor medio, categorías preferidas, recurrencia) con una propuesta de acción de marketing por segmento. Simulación de impacto de negocio. Servicios de segmentación y evaluación.
6	**Integración, panel y memoria.** Integración de los cinco servicios con Docker. Desarrollo del panel interactivo en Streamlit (ficha de cliente con CLV, segmento y recomendaciones, gráfico de segmentos y tabla comparativa de modelos). Pruebas de extremo a extremo, redacción de la memoria final con la justificación de las decisiones de diseño y la comparación Deep Learning vs. modelos clásicos, y preparación de la presentación.

6.1 Entregables

- Repositorio con el código de los cinco servicios y las instrucciones para ejecutarlo.
- Panel interactivo funcional sobre los datos de Olist.
- Memoria final con la metodología, los resultados y la discusión.
- Presentación del proyecto.

6.2 Riesgos y mitigación

Riesgo	Mitigación
CLV con muchos valores cero y poca señal	Comparar varios *targets* y transformaciones; incluir la probabilidad de recompra como alternativa.
La red no mejora a los modelos clásicos	Es un resultado válido: se documenta y se analiza por qué, lo que forma parte del objetivo de comparación.
Recomendación demasiado dispersa	Usar atributos y categoría del producto y evaluar también a nivel de categoría.
Retrasos en la integración	Cada servicio publica desde la semana 1 un *endpoint* con datos de ejemplo que respeta el contrato acordado.

7 Aportación a la asignatura

El proyecto responde directamente a la idea con la que la asignatura resume el papel del Machine Learning en marketing digital: *predecir comportamientos, clasificar clientes, personalizar acciones y optimizar decisiones*. Cada uno de estos cuatro verbos queda cubierto dentro de un único sistema construido sobre un mismo dato de partida (el historial de compras en Olist), lo que permite mostrar de forma tangible cómo un mismo dato puede responder a múltiples preguntas de marketing.

Además, al comparar de forma explícita los resultados de la red multi-tarea frente a los modelos clásicos presentados en la asignatura (Random Forest, Collaborative Filtering, K-Means), el proyecto aporta una base empírica para discutir en qué medida el Deep Learning mejora –o no– las decisiones de marketing digital, lo que consideramos de utilidad tanto para la evaluación práctica en grupo como para la preparación de la parte teórica del examen final.

La ampliación a seis semanas permite, además, cerrar el ciclo completo de un proyecto de datos aplicado al marketing: desde la exploración y la definición del problema de negocio hasta la evaluación rigurosa, la interpretación de los segmentos en términos de acciones de marketing y la entrega de una herramienta utilizable por una persona no técnica.